03 函數的成長

Part I Foundations

本章定位

以完整量詞、常數與反例建立五種漸近關係，區分輸入成本與函數界，再推導floor/mod、固定底數成長比較、factorial、log-star與Fibonacci的可用條件。

前置知識

實數、對數、多項式與基本不等式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用正式量詞定義 O、Omega 與 Theta。
- 證明多項式最高次項的漸近界限。
- 辨識對數底、指數與階乘分析中的常見等式。
- 3.1 Asymptotic notation
- 3.2 Standard notations and common functions

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.1 Asymptotic notation

先選定函數及極限方向。本節n取自然數並趨向∞；f最終非負、比較基準g最終嚴格正。若g可為0，須回到不等式定義，不能直接除以g。有限多個小n的異常可由門檻n_0排除，程式的合法小輸入仍須另保正確。

O(g)、Ω(g)、Θ(g)是**函數集合**；寫f=O(g)慣例表示f∈O(g)，不是數值相等。f可以是時間、空間、比較次數或純數學函數。定理只看函數，不自行指定它是最佳、最差還是平均成本。

例如insertion的最差函數Θ(n^2)，最佳函數Θ(n)。對所有輸入的共同上界O(n^2)不給所有輸入共同下界Ω(n^2)。函數與輸入量詞須同時交代。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三種界的量詞與常數

符號	精確定義：下式對所有整數$n \geq n_0$成立
$f \in O(g)$	存在$c>0, n_0$，使$0 \leq f(n) \leq c\, g(n)$
$f \in \Omega(g)$	存在$c>0, n_0$，使$0 \leq c\, g(n) \leq f(n)$
$f \in \Theta(g)$	存在$c_1, c_2>0, n_0$，使$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$

常數可依f/g及固定演算法而定，不能依n變。例如$f=n^2$、$g=n$，令$c=n$不能證$O(n)$。門檻不必最小；任何更大n_0都仍可用。緊界意味上下兩側都成立，不表示f/g必有極限，也不表示兩演算法同秒數。

$\Theta(g) \subseteq O(g)$且$\Theta(g) \subseteq \Omega(g)$。$n \in O(n^2)$是正確但寬鬆的界；把O當成Θ會丟失這個區分。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

一個有具體常數的雙界證明

取$f(n)=3n^2-8n+5$。$n\geq 4$時，$8/n\leq 2$、$5/n^2\geq 0$，所以$f/n^2=3-8/n+5/n^2\geq 1$；另一方面$n\geq 1$時$-8n+5\leq 0$，$f\leq 3n^2$。可選$c_1=1, c_2=3, n_0=4$，證$f\in\Theta(n^2)$。

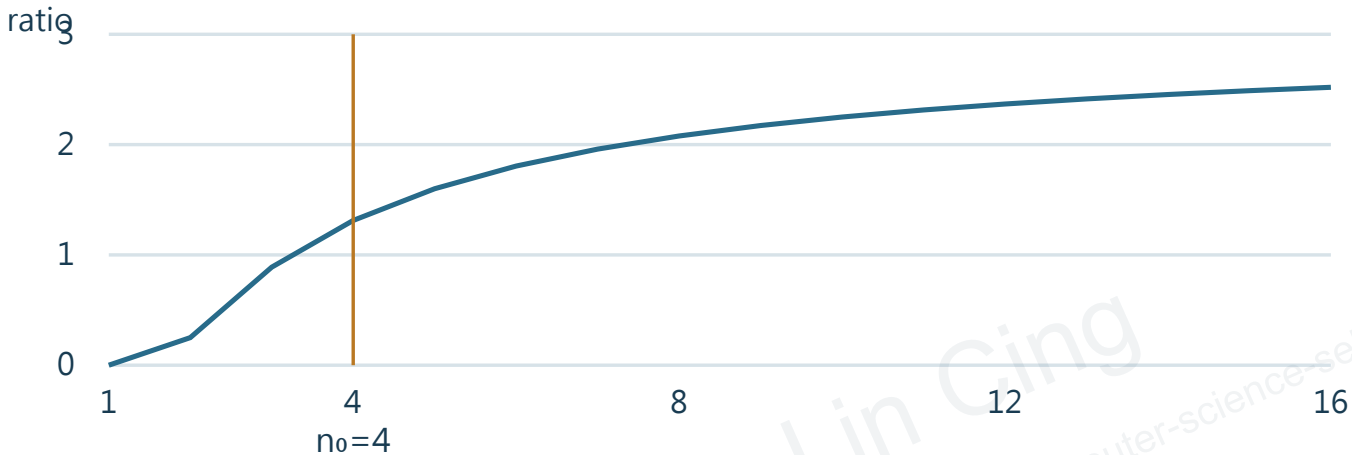
逐值檢查：$n=4$時$f=21$，$n^2=16$，$3n^2=48$；$n=8$時$f=133$，界$64\leq 133\leq 192$。有限表不是全稱證明；上段不等式才涵蓋所有$n\geq 4$。

反證$f\notin O(n)$：若$f\leq cn$且$n\geq 4$有$f\geq n^2$，則$n\leq c$。取整數$n>\max(n_0,4,c)$矛盾。反例不是只找一個超過某個猜測c的點，而是對**任意候選常數**都能找到更大的n。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一個有具體常數的雙界證明 (續)

雙界要控制整個尾段，而非只看幾個點



比值 $f(n)/n^2$

$f=3n^2-8n+5$

$n \geq 4$：$1 \leq$ 比值 ≤ 3

$c_1=1$；$c_2=3$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Θ等於O與Ω的交集

若f∈Θ(g)，直接取定義中的c_2證O，取c_1證Ω。反向若$f \le c_U g$對$n \ge N_U$、$f \ge c_L g$對$n \ge N_L$，取$n_0 = max(N_U, N_L)$，同時得到$c_L g \le f \le c_U g$，故f∈Θ(g)。

對最終非負f,g，max(f,g)≤f+g≤2max(f,g)，故f+g=Θ(max(f,g))。這個結論依非負性；若允許相消，f=n、g=−n會使和為0。

若$p(n) = a_d n^d + \sum_{i<d} a_i n^i$、$a_d > 0$且d≥1，令$M = \sum_{i<d} |a_i|$。$n \ge max(1, 2M/a_d)$時，低階絕對和≤$M n^{d-1} \le a_d n^d / 2$，得$(a_d/2) n^d \le p \le (3a_d/2) n^d$。d=0時為正定常數Θ(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

little-o與little-ω：把「存在c」改成「任意c」

$f \in o(g)$：對**每個**$c>0$，存在$n_0(c)$，使$0 \le f(n) < c\, g(n)$對所有$n \ge n_0$成立。$f \in \omega(g)$：對每個$c>0$，存在$n_0(c)$，使$0 \le c\, g(n) < f(n)$。門檻可以依c，但不能在固定c後再依n。

例如$7n \in o(n^2)$：給$c>0$，取整數$n_0 > 7/c$即可。$7n^2 \notin o(n^2)$：取$c=7$便要求$7n^2 < 7n^2$。$n^2 \in \omega(7n)$：給$c>0$，取$n>7c$。對正g，o等價$f/g \to 0$；對最終正f,g，ω等價$f/g \to \infty$。

$o(g) \subseteq O(g)$，但不是所有「非緊O界」都是o。$f=n$在偶數、$f=n^2$在奇數時，$f \in O(n^2)$卻不在$\Theta(n^2)$，也不在$o(n^2)$；其比值反覆回到1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

極限法何時能下結論

對最終正g，若L=lim f/g存在：L=0給o(g)；0<L<∞給Θ(g)；L=∞且f最終正給ω(g)。有限正L的證明：取ε=L/2，充分大時L/2<f/g<3L/2，即雙界。

極限不存在不能直接判「不可比較」。$f = n(2 + (-1)^n)$，g=n，比值為1或3，沒有極限卻有n≤f≤3n，仍為Θ(n)。

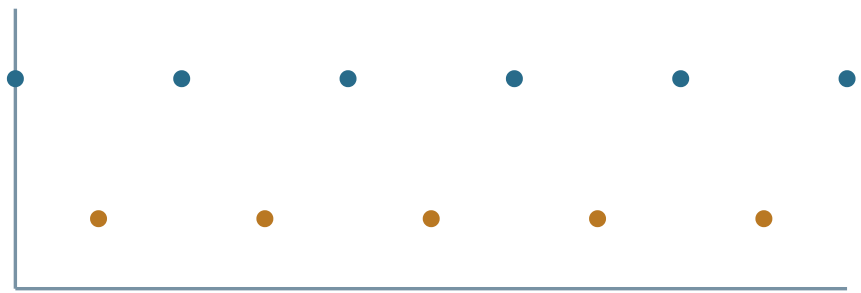
真正不可比較例：f=n^2（偶數n）、f=1（奇數n），g=n。偶數比值n無上界，故非O(g)；奇數比值1/n趨0，故非Ω(g)。取對應奇偶子序列便可逐一否定任意常數界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

極限法何時能下結論 （續）

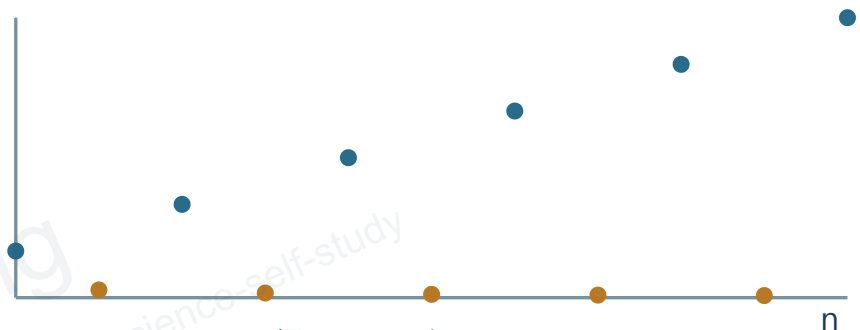
畫的是f/g：左圖夾在1與3，右圖兩個子序列分離

有Θ界，但無比值極限



偶n：3；奇n：1

既非O，也非Ω



偶n：n；奇n：1/n

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

關係的代數性質與證明

對最終正函數，O、Ω、Θ具反身與傳遞性，Θ具對稱性；f∈O(g)當且僅當g∈Ω(f)，f∈o(g)當且僅當g∈ω(f)。例如f≤ag、g≤bh，取共同門檻便有f≤ab h，證O傳遞；Ω同理反向，Θ兩側各做一次。

o傳遞也須對任意常數：給c>0，取充分大n使f<g（使用常數1），且g<c h，則f<c h。ω傳遞以倒置的o關係得到。o與ω不反身，且o(g)∩ω(g)=∅：同取c=1的兩個共同門檻，會要求f<g且g<f。

這些關係不是實數全序：前頁的奇偶例不符合O、Ω任一方向，因此不能假定任意兩函數一定有小於、同階、大於三者之一。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

式子中的匿名函數與不可逆等號

寫$4n^2+6n+2=4n^2+\Theta(n)$，表示選了某個$h\in\Theta(n)$，這裡$h=6n+2$。寫$4n^2+\Theta(n)=\Theta(n^2)$，表示**每個**左側h都可在右側找到相符的函數；不是左右集合完全相等，不能倒過來用。

兩個O項各可選不同匿名函數；$O(n)-O(n)$無法保證$\Theta(n)$，例如$n-n=0$。若要討論有號誤差，應明寫$|e(n)|\le cn$，因本章非負版本O集合不自動包含負函數。

$\sum_{i=1}^{n} h(i)$，若h(i)=O(i)，隱含同一c及門檻對所有充分大i有效；有限prefix和為常數，故和$O(n^2)$。若讓每項各自依n選不同常數，像$h_n(i)=n\cdot i$，就不符合這個單一h假設，總和可達$\Theta(n^3)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-st

對演算法輸入的界必須一致

令T(x)≥0，$T_{\max}(n) = \max_{|x|=n} T(x)$，$T_{\min}(n) = min_{|x|=n} T(x)$，假設各集合非空且極值存在。所有輸入共同Θ(g)的精確意思是存在同一c_L, c_U, n_0，使n≥n₀的**每個**x都滿足$c_L g(n) \le T(x) \le c_U g(n)$。

它等價$T_{\max} \in O(g)$且$T_{\min} \in \Omega(g)$：共同界取max/min得到一方向；反向以$T_{\min} \le T(x) \le T_{\max}$及共同門檻得到。只有$T_{\max} \in \Theta(g)$不夠，因它的下界只是某些最慢輸入。

若同時有n,m兩參數，須另約定量詞範圍。本章採n≥n₀且m≥m₀同時成立的矩形尾域；若要求「n或m足夠大」的更強尾域，需明寫or，不能無聲交換兩種定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.2 Standard notations and common functions

monotonically increasing（單調不減）：$m \le n \Rightarrow f(m) \le f(n)$；strictly increasing則$m<n \Rightarrow f(m)<f(n)$。decreasing對應反向不等式，不要把nondecreasing誤當必嚴格上升。

若f,g單調不減，f+g亦然；若g的值落在f的定義域，f∘g亦然。若再皆非負，$f(m)g(m) \le f(n)g(m) \le f(n)g(n)$，積也單調不減；沒有非負條件時不一定。

floor(x)是≤x的最大整數，ceiling(x)是≥x的最小整數，所以$x-1<\lfloor x \rfloor \le x \le \lceil x \rceil < x+1$。例$x=-2.3$：floor=−3、ceil=−2，不能把「截掉小數」當一般floor。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

整數分割、巢狀floor／ceiling與mod

整數n有ceil(n/2)+floor(n/2)=n；偶數2k給k+k，奇數2k+1給(k+1)+k。正整數a,b與x≥0有floor(floor(x/a)/b)=floor(x/(ab))；因對整數k，kb≤floor(x/a)等價kab≤x，兩式可接受的最大k相同。ceiling對最小k及反向界同理。

正整數a,b有ceil(a/b)≤(a+b−1)/b、floor(a/b)≥(a−b+1)/b，可由a=qb+r、0≤r<b分情況r=0／r>0證得。

對整數u與正模數m，u mod m=u−m floor(u/m)∈{0,⋯,m−1}。例−17 mod5=−17−5(−4)=3，17 mod5=2。程式語言若remainder符號跟dividend，不可直接當此數學mod。u≡v(mod m)當且僅當m整除u−v。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

冪次、指數與常數的範圍

對a>0及實數u,v，$a^{u+v} = a^u a^v$，$(a^u)^v = a^{uv}$，a⁰=1、$a^{-u} = 1/a^u$。本章a為**固定常數**時，a>1使a^n增加，0<a<1使其減少。n^b對n>0，b>0增加、b<0減少。

polynomially bounded是存在固定k使$f = O(n^k)$，不只指f本身是多項式。$n^{\lg n}$不符合：對任意固定k，$f/n^k = n^{\lg n - k} \to \infty$。$2^{\lg n} = n$是多項式，但$2^n$是指數；看到符號2不代表函數一定指數成長。

0⁰若出現組合公式須另外定義，本章在空乘積慣例下採1，不主張它是每個極限的共同值。實數底數、指數的恆等式也不能無條件搬到負底數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼固定底數指數超過任何多項式

設a=1+δ>1、b為固定實數。若b≤0，$n^b/a^n \le 1/a^n \to 0$。若b>0，選整數k>b。二項式展開給$a^n \ge binom(n,k)\delta^k$。

n≥2k時，$n(n-1)\dots(n-k+1) \ge (n/2)^k$，所以$a^n \ge \delta^k n^k/(2^k k!)$。因此$0 \le n^b/a^n \le (2^k k!/\delta^k)n^{b-k} \to 0$，得$n^b = o(a^n)$。k依b但不依n。

對固定ε>0及任意固定b，令t=lg n，則$(\lg n)^b/n^\varepsilon = t^b/(2^\varepsilon)^t \to 0$。因此任何固定polylog都小於每個正次冪。底數若隨n接近1，就不符合固定a>1前提；例如$(1+1/n)^n$趨e。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對數的底數、次方與複合不可混

lg n=log₂n，$\ln n = log_e n$；$\lg^k n = (\lg n)^k$，lg lg n=lg(lg n)，$\lg^{(k)} n$表示反覆取lg共k次。log只作用於緊鄰項，lg n+q不是lg(n+q)。所有實數log的引數須>0，底數>0且≠1。

由$b^{log_b x} = x$，取log_c得$log_b x = log_c x / log_c b$。固定底b,c>1只差正定常數，所以$log_b n = \Theta(log_c n)$。若b=n，$log_n n = 1$，與lg n不同階。

$log_b(xy) = log_b x + log_b y$、$log_b(x^r) = r log_b x$；$a^{log_b c} = c^{log_b a}$，因兩邊取ln都為(ln a ln c)/(ln b)。例$3^{lg 8} = 27 = 8^{lg 3}$，不是$3^{(\lg)^8}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

e與ln的可用界及誤差

$e^x = \sum_{k\ge 0} x^k/k!$；由凸性（導數$e^x$遞增，x=0切線斜率1）得$e^x \ge 1+x$，全實數成立。Taylor積分／Lagrange餘項給$e^x = 1 + x + (e^\xi/2)x^2$，ξ介於0與x，所以|x|≤1時$e^{-1}x^2/2 \le e^x - 1 - x \le ex^2/2$，證x→0時餘項Θ(x²)。

對x>−1，x/(1+x)≤ln(1+x)≤x。上界可用h=x−ln(1+x)，h(0)=0且h'=x/(1+x)，在0兩側都使h≥0；下界用q=ln(1+x)−x/(1+x)，q'=x/(1+x)²同理。若|x|<1，$\ln(1+x) = \sum_{k\ge 1}(-1)^{k+1}x^k/k$。

固定x，令u=x/n，當n大到1+u>0，兩個ln界夾出n ln(1+x/n)→x，再取exp，得$(1+x/n)^n \to e^x$。這些是有定義域的界，不是把所有近似式當成恆等式。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Factorial：先用乘積得到一般界

整數n≥0，0!=1，n!=n(n−1)!。每個乘數≤n，故$n! \le n^n$。更強地，前floor(n/2)個乘數≤n/2，剩餘≤n，所以$n!/n^n \le 2^{-\operatorname{floor}(n/2)} \to 0$，證$n! = o(n^n)$。

至少floor(n/2)個尾端乘數≥n/2，故$n! \ge (n/2)^{\operatorname{floor}(n/2)}$。取lg後，lg(n!)介於floor(n/2)lg(n/2)與n lg n，得Θ(n lg n)。同一下界扣去n lg a，對任意固定a>0皆趨∞，所以$n!/a^n \to \infty$；尤其$n! = \omega(2^n)$。

這些界不需先假設Stirling。數例6!=720、2^6=64、6^6=46656；一個有限數例不能代替上述極限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stirling的精度與指數中的常數

Stirling界（此處陳述標準近似定理）：n≥1時

$n! = \mathrm{sqrt}(2\pi n)(n/e)^n e^{\alpha_n}$，$1/(12n+1) < \alpha_n < 1/(12n)$。

因此相對誤差因子$e^{\alpha_n} = 1 + \Theta(1/n)$，且lg(n!)=n lg n−(lg e)n+(1/2)lg(2πn)+Θ(1/n)。n=6的主項$\sqrt{12\pi}(6/e)^6$約710.078，乘上介於$e^{1/73}$與$e^{1/72}$的因子夾住720。

「log底數只差常數」不能推成指數底數也可忽略。$2^{n+3} = 8 \cdot 2^n$，故$\Theta(2^n)$；$2^{3n}/2^n = 4^n \to \infty$，不在$O(2^n)$。若f=Θ(n)，只能說$2^f = 2^{\Theta(n)}$，不能說必為$\Theta(2^n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

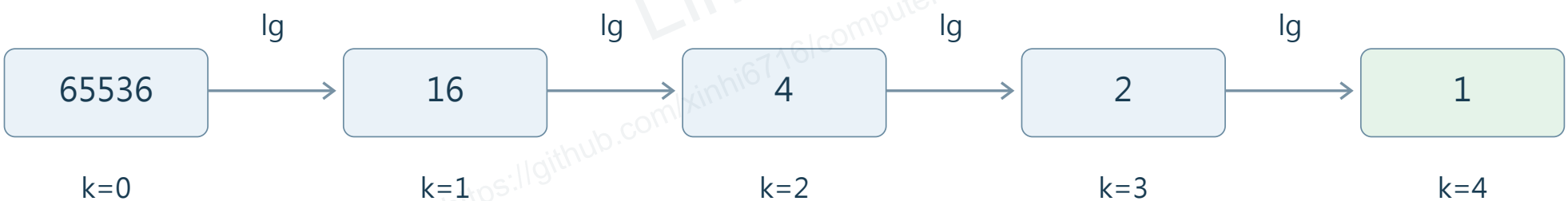
Functional iteration與log-star的停止點

$f^{(0)}(n) = n$，$f^{(k+1)}(n) = f(f^{(k)}(n))$。若f(n)=3n，則$f^{(k)}(n) = 3^k n$，不是$(3n)^k$。

對n>0，$\lg^* n = \min\{k \geq 0 : \lg^{(k)} n \leq 1\}$；只在前一值>1時繼續取lg，因此不會讀log非正數。n≤1時lg*=0。例如65536→16→4→2→1，故lg*65536=4；$\lg^{(2)} 65536 = 4$，而lg²65536=256。

令tower門檻a_0=1、$a_{k+1} = 2^{a_k}$，則$n \in (a_{k-1}, a_k]$時lg*n=k（k≥1）。1,2,4,16,65536依序是k=0..4的上界。log-star雖慢仍無界：每個有限k都有更大n超過a_k。

反覆取lg：每個箭頭都算一次，不再對1取lg



只在目前值>1時繼續；第4次得到1，立即停止，所以lg*65536=4。

連取兩次lg得4；(lg65536)²=256，iteration與power是兩種不同運算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci閉式不是猜到就算證完

F₀=0、F₁=1、$F_n = F_{n-1} + F_{n-2}$。設φ=(1+sqrt5)/2、ψ=(1−sqrt5)/2，兩者都滿足z²=z+1。令$G_n = (\varphi^n - \psi^n)/\sqrt{5}$；G₀=0、G₁=1，且對n≥2，$\varphi^n = \varphi^{n-1} + \varphi^{n-2}$、ψ也同理，故$G_n = G_{n-1} + G_{n-2}$。由歸納$F_n = G_n$。

|ψ|<1給$|F_n - \varphi^n/\sqrt{5}| = |\psi|^n/\sqrt{5} < 1/2$（含n=0），故$F_n = \mathrm{floor}(\varphi^n/\sqrt{5} + 1/2)$。n=6時φ⁶/sqrt5≈8.024922，ψ⁶/sqrt5≈0.024922，相減=8。

由誤差<1/2且φ^n無界，充分大n有$\varphi^n/(2\sqrt{5}) \le F_n \le 2\varphi^n/\sqrt{5}$，故$F_n = \Theta(\varphi^n)$。閉式實數運算不代表能用有限精度浮點O(1)精確輸出任意大F_n；其輸出已有Θ(n) bits。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 03。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study